

PRACA DYPLOMOWA MAGISTERSKA

Analiza wydajności i efektywności wybranych algorytmów sortowania równoległego na wielordzeniowych procesorach

*A performance and efficiency analysis of selected
parallel sorting algorithms on multicore processors*

Krzysztof Pielot

Nr albumu: 133537

Kierunek: Informatyka

Forma studiów: stacjonarne

Poziom studiów: II

Promotor pracy:

dr inż. Bartosz Kowalczyk

Praca przyjęta dnia:

Podpis promotora:

Częstochowa, 2026

Niniejszym chciałbym serdecznie podziękować

...

*za bezcenne wsparcie udzielone mi
w trakcie trwania studiów.*

Spis treści

Wstęp	3
Cel pracy	4
Zakres pracy	4
1 Wprowadzenie teoretyczne	7
1.1 Wprowadzenie do sortowania równoległego	7
1.2 Charakterystyka procesorów wielordzeniowych	7
1.3 Wydajność i efektywność	7
1.4 Skalowalność algorytmów równoległych	7
2 Opis wybranych algorytmów sortowania równoległego	9
2.1 Parallel Quicksort	9
2.2 Parallel Merge Sort	9
2.3 Parallel Bucket Sort	9
2.4 Parallel Samplesort	9
3 Środowisko badawcze i implementacja	10
3.1 Opis środowiska testowego	10
3.1.1 Konfiguracje sprzętowe	10
3.1.2 Konfiguracje systemowe	10
3.2 Wybór języka i bibliotek	10
3.3 Sposób implementacji algorytmów	10
3.4 Charakterystyka danych testowych	11
4 Metodyka badań	12
4.1 Scenariusze testowe	12
4.2 Wariant sekwencyjny jako punkt odniesienia	12
4.3 Parametry pomiarów	12
4.4 Narzędzia do pomiaru czasu, CPU i pamięci	12
4.5 Przebieg eksperymentów	12
5 Analiza wyników badań	13
5.1 Porównanie czasu sortowania	13

5.2	Wpływ liczby rdzeni na wydajność	13
5.3	Analiza zużycia pamięci RAM i obciążenia CPU	13
5.4	Skalowalność algorytmów	13
5.5	Wpływ typu danych	13
5.6	Porównanie efektywności w zależności od charakterystyki danych wejściowych	14
Podsumowanie		15
Bibliografia		17
Spis rysunków		18
Spis tabel		19
Listing		20
Streszczenie		21
Summary		22
Słowa kluczowe		23

Wstęp

W ostatnim czasie można zaobserwować intensywny rozwój architektury procesorów wielordzeniowych. Producenci sprzętu zamiast dalszego zwiększania częstotliwości taktowania pojedynczego rdzenia decydują się na zwiększanie liczby rdzeni fizycznych oraz logicznych. Taka zmiana podejścia projektowania procesorów stawia przed twórcami oprogramowania nowe wyzwania. Klasyczne algorytmy sekwencyjne nie wykorzystują w pełni dostępnej mocy obliczeniowej współczesnych jednostek centralnych.

Szczególnie ważnym obszarem, w którym równoległość może przynieść wymierne korzyści, jest sortowanie danych. Sortowanie stanowi jeden z fundamentalnych problemów informatyki i jest wykorzystywane w ogromnej liczbie miejsc takich jak bazy danych, systemy plików, aż po przetwarzanie dużych zbiorów danych i uczenie maszynowe. Wraz ze wzrostem objętości przetwarzanych informacji klasyczne algorytmy sortowania sekwencyjnego mogą coraz częściej stać się wąskim gardłem wydajnościowym.

Niestety, implementacja efektywnych algorytmów sortowania równoległego nie jest zadaniem trywialnym. Wymaga ona odpowiedniego podziału danych, synchronizacji procesów, zarządzania pamięcią współdzieloną oraz minimalizacji narzutu związanego z tworzeniem i komunikacją procesów. W praktyce wiele istniejących rozwiązań albo pozostaje na poziomie teoretycznym, albo nie uwzględnia realnych ograniczeń sprzętowych i systemowych, takich jak koszt przełączania kontekstu, konflikty w dostępie do pamięci czy nierównomierne obciążenie rdzeni.

W związku z tym, interesującym i aktualnym zagadnieniem wydaje się praktyczne zbadanie wydajności oraz efektywności wybranych algorytmów sortowania równoległego w rzeczywistych warunkach obliczeniowych. Niniejsza praca koncentruje się na implementacji i porównaniu czterech popularnych algorytmów: Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort. Algorytmy zostały zrealizowane w języku Python z wykorzystaniem biblioteki `multiprocessing`, a następnie poddane systematycznym testom pod kątem czasu wykonania, zużycia pamięci RAM, obciążenia procesora, skalowalności oraz wpływu charakterystyki danych wejściowych.

Cel pracy

Głównym celem niniejszej pracy magisterskiej jest przeprowadzenie kompleksowej analizy wydajności i efektywności wybranych algorytmów sortowania równoległego na współczesnych wielordzeniowych procesorach. Praca skupia się na praktycznej implementacji oraz systematycznym porównaniu czterech algorytmów: Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort.

Celem szczegółowym jest zbadanie, w jaki sposób wybrane algorytmy zachowują się w realnych warunkach obliczeniowych, a nie jedynie w warunkach idealnych zakładanych przez modele teoretyczne. W tym celu algorytmy zostały zaimplementowane w języku Python z wykorzystaniem biblioteki `multiprocessing`, która umożliwia tworzenie procesów oraz współdzielenie pamięci. Każdy z algorytmów został uruchomiony zarówno w wariancie sekwencyjnym stanowiącym punkt odniesienia, jak i w wariancie równoległym, przy różnej liczbie wykorzystywanych rdzeni procesora.

Badanie obejmuje kilka kluczowych aspektów:

- ▶ pomiar czasu sortowania w zależności od rozmiaru zbioru danych oraz liczby użytych rdzeni,
- ▶ analizę zużycia pamięci operacyjnej (RAM) oraz obciążenia procesora (CPU) podczas wykonywania algorytmów,
- ▶ ocenę skalowalności, czyli zdolności algorytmów do efektywnego wykorzystania rosnącej liczby rdzeni,
- ▶ porównanie zachowania algorytmów przy różnych charakterystykach danych wejściowych (dane losowe, częściowo posortowane, z duplikatami).

Dodatkowym celem pracy jest wyznaczenie i analiza klasycznych metryk równoległości, *speedup* (przyspieszenie) oraz *efficiency* (efektywność), a także identyfikacja głównych czynników ograniczających wydajność poszczególnych algorytmów (m.in. narzut związany z tworzeniem procesów, synchronizacją, komunikacją oraz zarządzaniem pamięcią współdzieloną).

W rezultacie pracy oczekuje się uzyskania praktycznych wniosków dotyczących tego, które z badanych algorytmów sortowania równoległego najlepiej sprawdzają się w środowisku wielordzeniowym przy określonych rozmiarach i typach danych, a także wskazania ich mocnych i słabych stron w kontekście rzeczywistego wykorzystania zasobów obliczeniowych.

Zakres pracy

Zakres pracy obejmuje:

- ▶ zebranie wiadomości z zakresu sortowania równoległego, architektury procesorów wielordzeniowych, metryk wydajności i efektywności oraz skalowalności algorytmów równoległych,
- ▶ porównanie wybranych algorytmów sortowania równoległego (Parallel Quicksort, Parallel MergeSort, Parallel BucketSort oraz Parallel Samplesort) pod kątem wydajności, skalowalności i zużycia zasobów obliczeniowych,
- ▶ opracowanie założeń projektowych dotyczących implementacji algorytmów w środowisku wielordzeniowym z wykorzystaniem biblioteki `multiprocessing` oraz pamięci współdzielonej,
- ▶ implementację wariantów sekwencyjnych i równoległych wybranych algorytmów sortowania w języku Python,
- ▶ implementację generatora danych testowych o różnych charakterystykach (dane losowe, częściowo posortowane, odwrotnie posortowane oraz dane o określonym rozkładzie),
- ▶ przetestowanie zaimplementowanych algorytmów na wielordzeniowych procesorach, obejmujące pomiar czasu wykonania, zużycia pamięci RAM, obciążenia CPU, wyznaczenie metryk *speedup* i *efficiency* oraz analizę wpływu charakterystyki danych wejściowych.
- ▶ prezentację wyników badań za pomocą tabel oraz wykresów.

W rozdziale 1 przedstawiono podstawy teoretyczne sortowania równoległego, charakterystykę procesorów wielordzeniowych oraz metryki wykorzystywane do oceny wydajności i skalowalności. W rozdziale 2 opisano wybrane algorytmy sortowania równoległego. W rozdziale 3 scharakteryzowano środowisko badawcze, zastosowane technologie oraz sposób implementacji algorytmów. W rozdziale 4 przedstawiono metodykę badań, scenariusze testowe oraz narzędzia pomiarowe. W rozdziale 5 zaprezentowano i przeanalizowano wyniki przeprowadzonych eksperymentów. Pracę zamyka podsumowanie zawierające najważniejsze wnioski oraz możliwe kierunki dalszych badań.

Rozdział 1

Wprowadzenie teoretyczne

1.1 Wprowadzenie do sortowania równoległego

Wprowadzenie w temat sortowania równoległego, dlaczego, kiedy oraz jakie korzyści. Jakie są ogólne strategie sortowania równoległego, czyli podział danych ich przetwarzanie oraz synchronizacja. Wyzwania z takiego podejścia jak komunikacja między wątkami, synchronizacja, narzut systemowy

1.2 Charakterystyka procesorów wielordzeniowych

Wyjaśnienie na jakiej architekturze sprzętowej będą testowane dane algorytmy, czyli czym jest procesor wielordzeniowy, jaka są różnice między rdzeniem fizycznym a wątkiem logicznym, jak przebiega komunikacja między rdzeniami, współdzielenie pamięci RAM, znaczenie równoległości sprzętowej i jej ograniczenia oraz wpływ liczby rdzeni na możliwości przyspieszania obliczeń

1.3 Wydajność i efektywność

Przedstawienie metryk używanych w analizie porównawczej algorytmów. Definicje wydajności oraz efektywności, czas przetwarzania, pomiary zużycia RAM, obciążenie CPU, Speedup oraz Efficiency

1.4 Skalowalność algorytmów równoległych

Omówienie, jak zmienia się wydajność algorytmu w zależności od liczby rdzeni oraz wielkości danych. Czym w ogóle jest skalowalności w informatyce, jakie są typy skalowalności czyli pozioma oraz pionowa, czynniki ograniczające skalowalność takie

jak synchronizacja, nierównomierny podział danych, dostęp do pamięci, zależności danych

Rozdział 2

Opis wybranych algorytmów sortowania równoległego

Dział poświęcony wybranym algorytmom zawierający opis, schemat działania, sposób dzielenia i łączenia, złożoność, potencjalne problemy oraz kiedy algorytm jest efektywny, a kiedy nie

2.1 Parallel Quicksort

2.2 Parallel Merge Sort

2.3 Parallel Bucket Sort

2.4 Parallel Samplesort

Rozdział 3

Środowisko badawcze i implementacja

3.1 Opis środowiska testowego

3.1.1 Konfiguracje sprzętowe

Szczegółowe dane dotyczące testowych maszyn takie jak model procesora, liczba rdzeni fizycznych i logicznych, pamięć RAM itp.

3.1.2 Konfiguracje systemowe

Szczegółowe dane dotyczące testowych maszyn takie jak system operacyjny, wersje sterowników i bibliotek systemowych itp.

3.2 Wybór języka i bibliotek

Informacje o wybranym języku programowania oraz bibliotek, uzasadnienie wyboru oraz opis

3.3 Sposób implementacji algorytmów

Opisanie w jaki sposób zostały zaimplementowane algorytmy sortujące. Ogólna architektura, w zależności od algorytmu jak następuje dzielenie, synchronizacja danych, wymiany danych, informacje jak został zaimplementowany waraint sekwencyjny

3.4 Charakterystyka danych testowych

Sposób przechowywania, typy danych(int, float), charakterystyka danych(losowe, częściowo posortowane, z dużą ilością duplikatów), rozmiar testowanych zbiorów danych, jak zostały wygenerowane dane do testów

Rozdział 4

Metodyka badań

4.1 Scenariusze testowe

Opis różnych warunków i przypadków testowych takich jak na jakich rozmiar danych, wariant algorytmu, liczba rdzeni, różne typy zbiorów(losowe itp.)

4.2 Wariant sekwencyjny jako punkt odniesienia

Wyjaśnienie, że dla porównania wyników algorytmów równoległych testowana jest także wersja sekwencyjna każdego algorytmu.

4.3 Parametry pomiarów

Parametry pomiarów co i jak będzie mierzone

4.4 Narzędzia do pomiaru czasu, CPU i pamięci

Opis wykorzystanych narzędzi i bibliotek użytych do pomiarów

4.5 Przebieg eksperymentów

Opis jak zostały przeprowadzone testy. Przygotowanie środowisk testowych. Sposób zapisywania i archiwizacji wyników. Procedura uruchomienia algorytmu itp.

Rozdział 5

Analiza wyników badań

5.1 Porównanie czasu sortowania

Prezentacja wyników pomiarów czasu wykonania dla każdego z algorytmów (dla różnych zbiorów danych, rozmiarów i typów danych). Omówienie i analiza

5.2 Wpływ liczby rdzeni na wydajność

Prezentacja i omówienie wyników pod względem użytych liczby rdzeni

5.3 Analiza zużycia pamięci RAM i obciążenia CPU

Prezentacja i omówienie zużycia pamięci i obciążenia w tym zestawienie średniego i maksymalnego zużycia

5.4 Skalowalność algorytmów

Ocena, jak dobrze algorytmy skalują się wraz ze wzrostem liczby rdzeni i rozmiarem danych

5.5 Wpływ typu danych

Porównanie wyników dla tych samych algorytmów, ale różnych typów danych wejściowych

5.6 Porównanie efektywności w zależności od charakterystyki danych wejściowych

Omówienie wyników dla danych losowych itp. Wskazanie, który algorytm radził sobie lepiej przy danym rodzaju danych

Podsumowanie

Bibliografia

- [1] Zasady pisania prac dyplomowych, <https://wiisi.pcz.pl/student-wiisi/vademecum-studenta/praca-dyplomowa>, stan na dzień: 26.10.2024
- [2] Strona internetowa Wydziału Informatyki i Sztucznej Inteligencji, <https://wiisi.pcz.pl>, stan na dzień: 26.10.2024

Spis rysunków

Spis tabel

Spis listingów

Streszczenie

Streszczenie

Summary

Tłumaczenie

Słowa kluczowe

informatyka; sortowanie równoległe; algorytmy sortowania; wielordzeniowe procesory; Python; wydajność algorytmów; skalowalność; pomiar czasu wykonania; zużycie pamięci RAM; obciążenie CPU; porównanie algorytmów; dane losowe; dane częściowo posortowane; duplikaty danych;

Do niniejszej pracy dołączono płytę z następującą zawartością:

- ▶ Dokument pracy w formatach tex i pdf.
- ▶ Kod źródłowy zaprojektowanego i zrealizowanego w ramach pracy systemu.

Imię i nazwisko: Krzysztof Pielot

Nr albumu: 133537

Kierunek: Informatyka

Wydział: Informatyki i Sztucznej Inteligencji

Oświadczenie autora pracy dyplomowej*

Oświadczam pod rygorem odpowiedzialności karnej, że złożona przeze mnie praca dyplomowa pt. „*Analiza wydajności i efektywności wybranych algorytmów sortowania równoległego na wielordzeniowych procesorach*” jest moim samodzielnym opracowaniem i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami.

Jednocześnie oświadczam, że moja praca (w całości ani we fragmentach) nie była wcześniej przedmiotem procedur związanych z uzyskaniem tytułu zawodowego w Politechnice Częstochowskiej.

Wyrażam/~~nie wyrażam~~** zgodę/zgody na nieodpłatne wykorzystanie przez Politechnikę Częstochowską całości lub fragmentów wyżej wymienionej pracy w publikacjach Politechniki Częstochowskiej.

.....
podpis studenta

*W przypadku zbiorowej pracy dyplomowej, dołącza się oświadczenia każdego ze współautorów pracy dyplomowej.

*Niepotrzebne skreślić.